Getting Started

VS Code + UV Setup

Live Demo: From Zero to First Project

What We'll Cover

Step-by-step setup:

1. 📥 Install VS Code
2. 🧩 Add Python extensions
3. ⚡ Install UV (package manager)
4. 🚀 Create your first project

Goal: By the end, you'll have a working Python development environment!

Part 1: Installing VS Code

Step 1: Download VS Code

Go to: https://code.visualstudio.com/

Click: Download for Windows/Mac/Linux

Step 2: Run Installer

Windows:

1. Run the downloaded `.exe` file
2. Accept license agreement
   ✅
3. **Check:** "Add to PATH" (important!)
4. Click Install

Mac: Drag to Applications folder

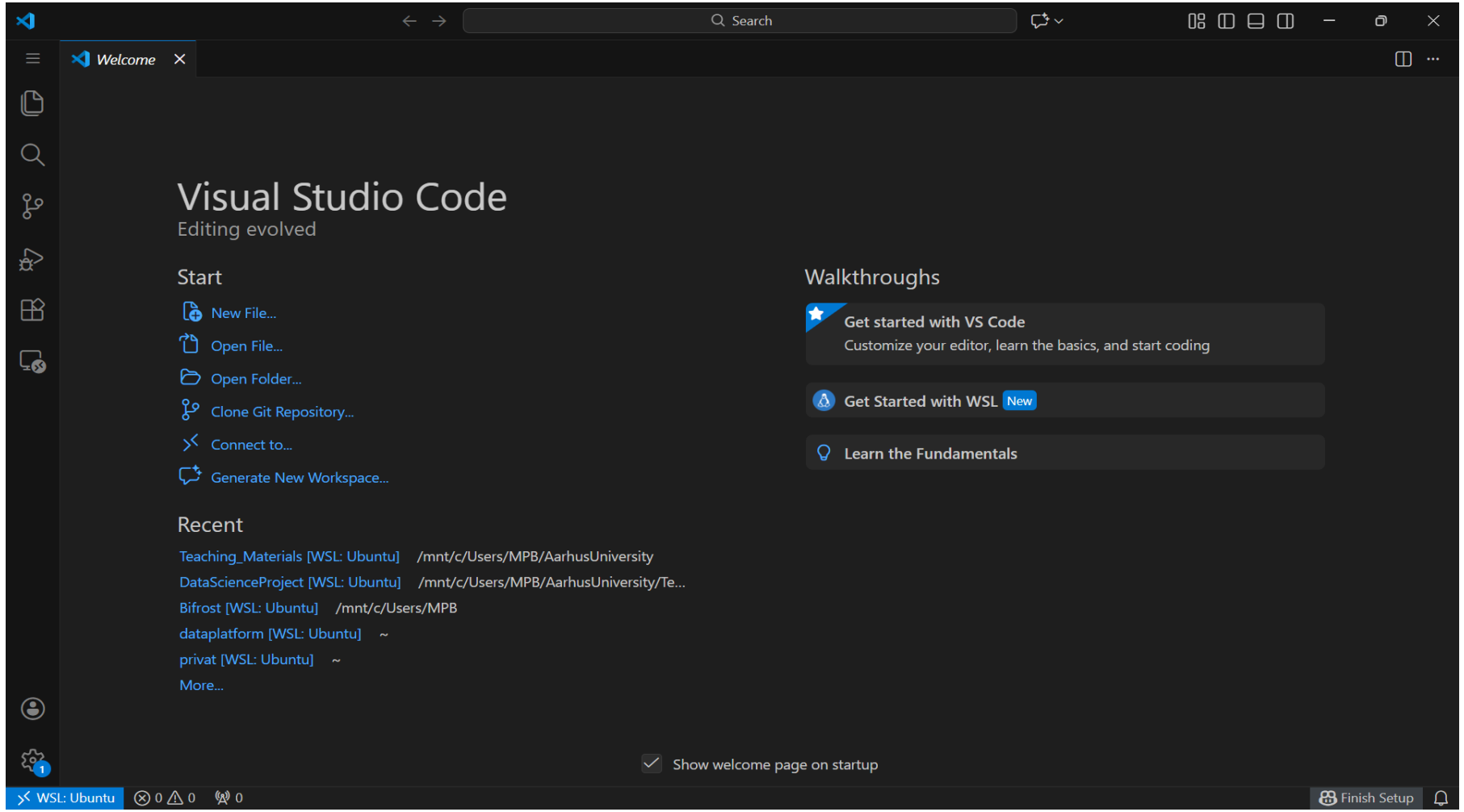
Linux: Follow package manager instructions

Step 3: Launch VS Code

First launch:

You'll see:

- Welcome screen
- Activity bar (left sidebar)
- Empty editor area

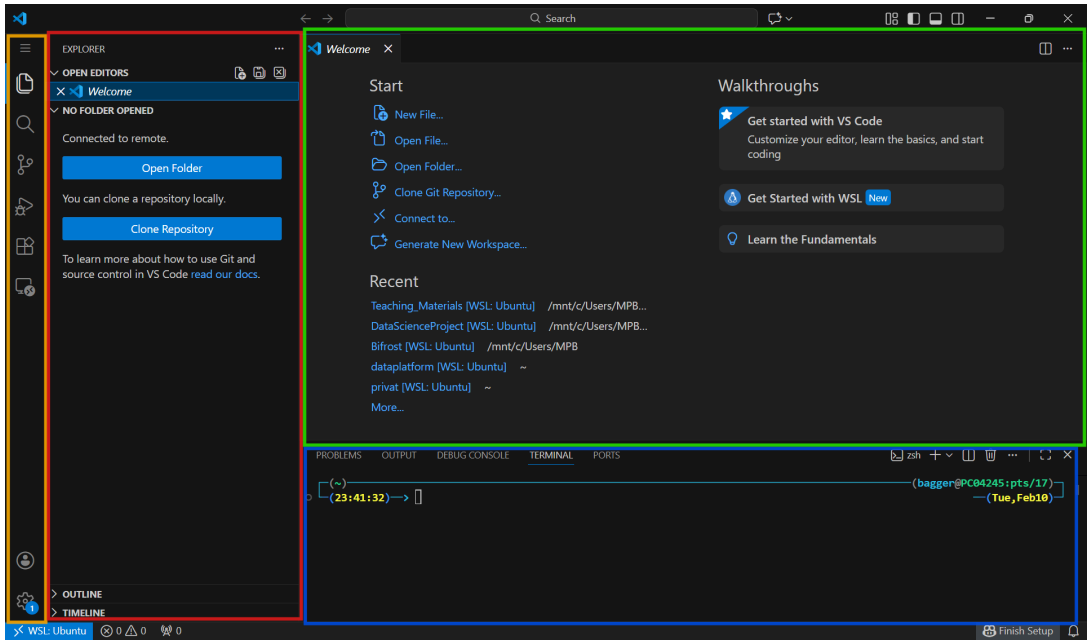

Search
Welcome
Visual Studio Code
Editing evolved
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Connect to...
Generate New Workspace...
Recent
Teaching_Materials [WSL: Ubuntu] /mnt/c/Users/MPB/AarhusUniversity
DataScienceProject [WSL: Ubuntu] /mnt/c/Users/MPB/AarhusUniversity/Te...
Bifrost [WSL: Ubuntu] /mnt/c/Users/MPB
dataplatform [WSL: Ubuntu] ~
privat [WSL: Ubuntu] ~
More...
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Get Started with WSL New
Learn the Fundamentals
Show welcome page on startup
WSL: Ubuntu
0 0
0
Finish Setup

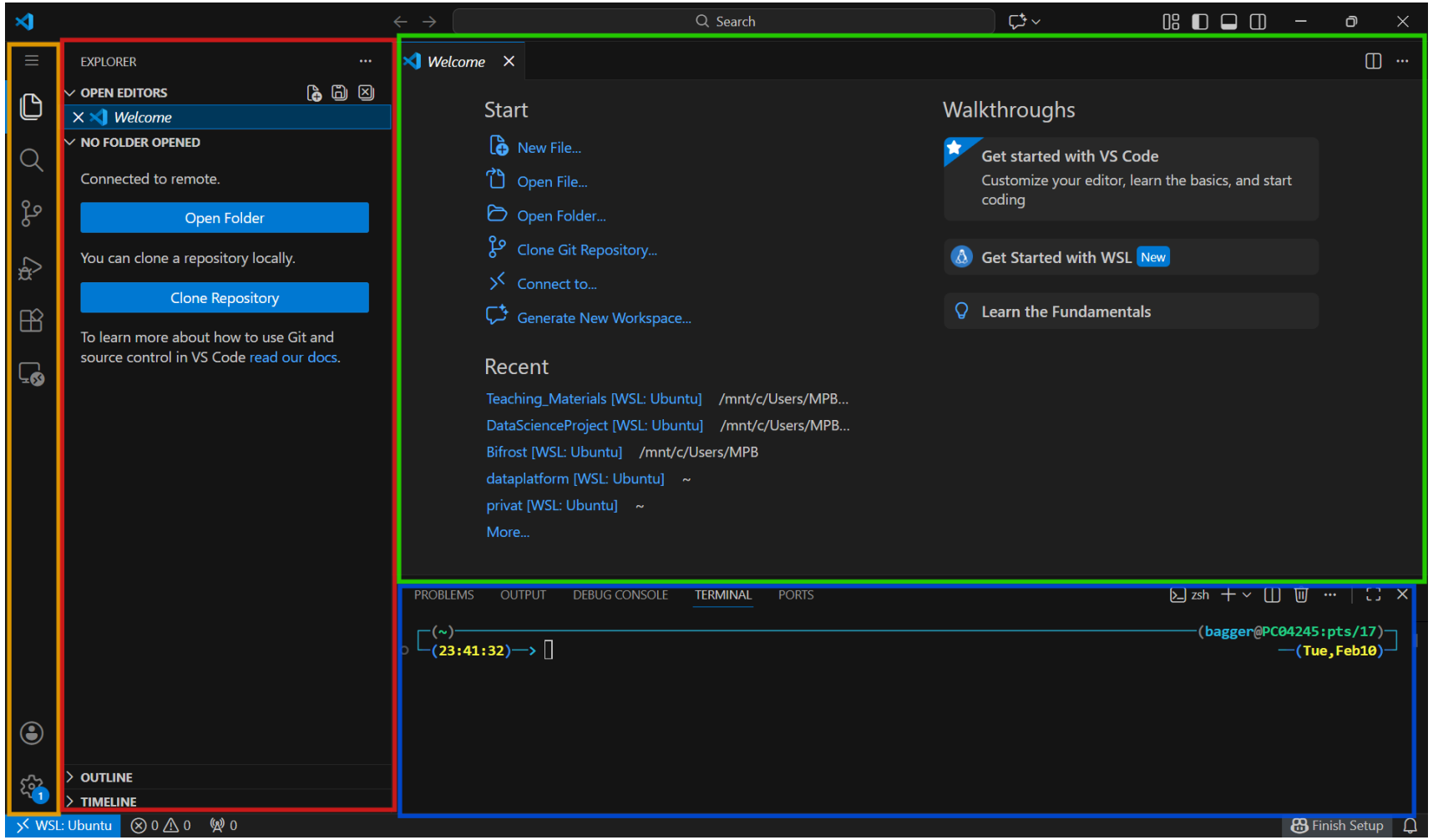
Part 2: Adding Extensions

Understanding VS Code Layout

Key areas we'll use:

1. **Activity Bar** (orange) - Navigate between views
2. **Side Bar** - (red) File Explorer, Search, Extensions
3. **Editor** - (green) Where you write code
4. **Terminal** - (blue) Command line interface (bottom)




Search
EXPLORER
OPEN EDITORS
Welcome
NO FOLDER OPENED
Connected to remote.
Open Folder
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
OUTLINE
TIMELINE
Welcome
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Connect to...
Generate New Workspace...
Recent
Teaching_Materials [WSL: Ubuntu] /mnt/c/Users/MPB...
DataScienceProject [WSL: Ubuntu] /mnt/c/Users/MPB...
Bifrost [WSL: Ubuntu] /mnt/c/Users/MPB
dataplatform [WSL: Ubuntu] ~
privat [WSL: Ubuntu] ~
More...
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Get Started with WSL New
Learn the Fundamentals
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
(~)
(23:41:32)
(bagger@PC04245:pts/17)
(Tue,Feb10)
WSL: Ubuntu
0 0
0
Finish Setup

Opening Extensions Panel

Click Extensions icon (or press `Ctrl+Shift+X` / `Cmd+Shift+X`)

This is where we install tools!

Install: Python Extension

Search: "Python"

Install: Python by Microsoft

This gives you:

- Python syntax highlighting
- Code completion
- Debugging
- Jupyter notebook support

Install: Pylance (Auto-installs with Python)

Pylance provides:

- Fast IntelliSense
- Type checking
- Auto-imports

Should install automatically with Python extension!

Recommended: Additional Extensions

Search and install:

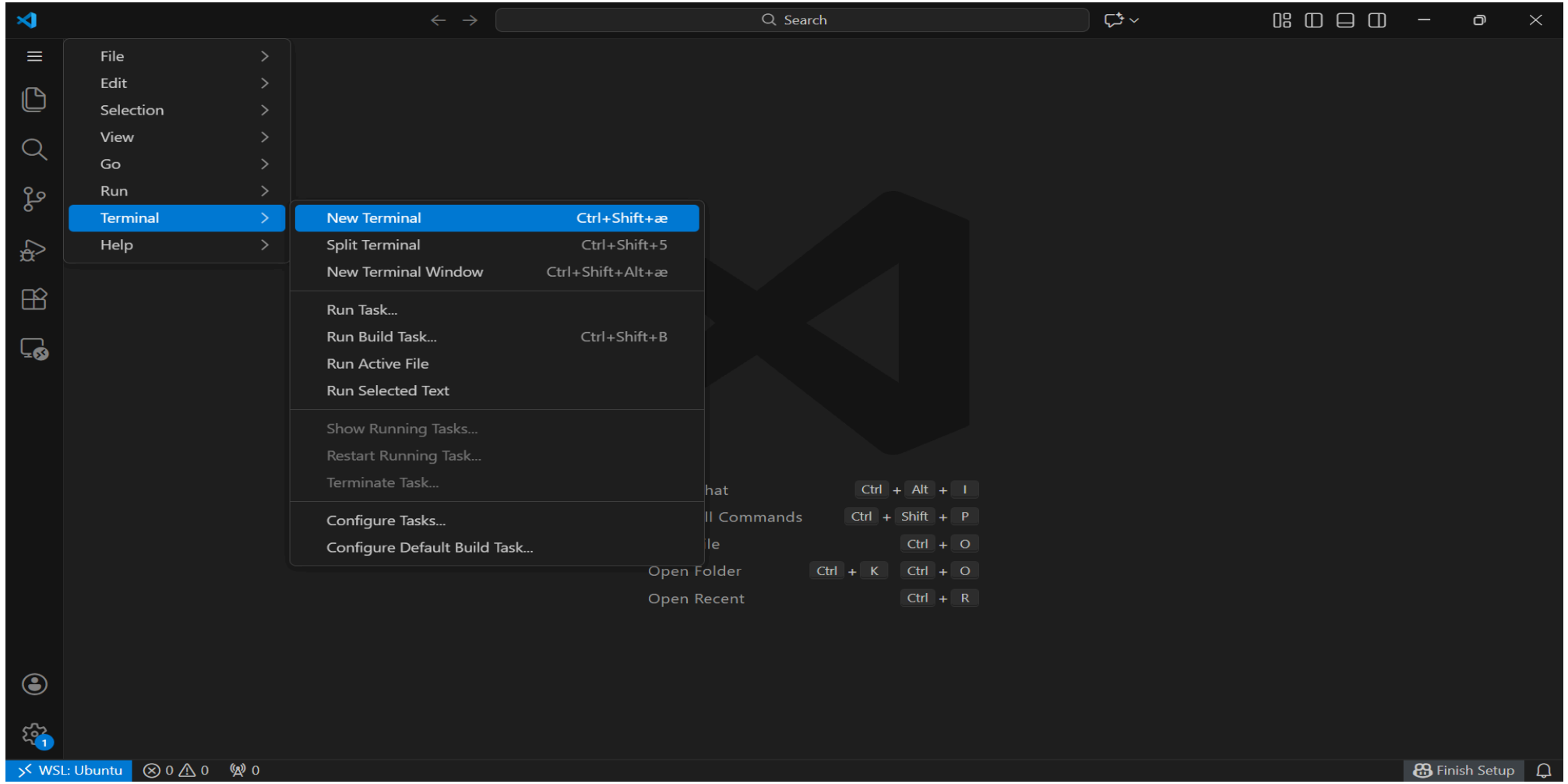
1. **Jupyter** - For notebooks
2. **Ruff** - Fast linter/formatter
3. **GitLens** - Git integration (optional)

Part 3: Opening the Terminal

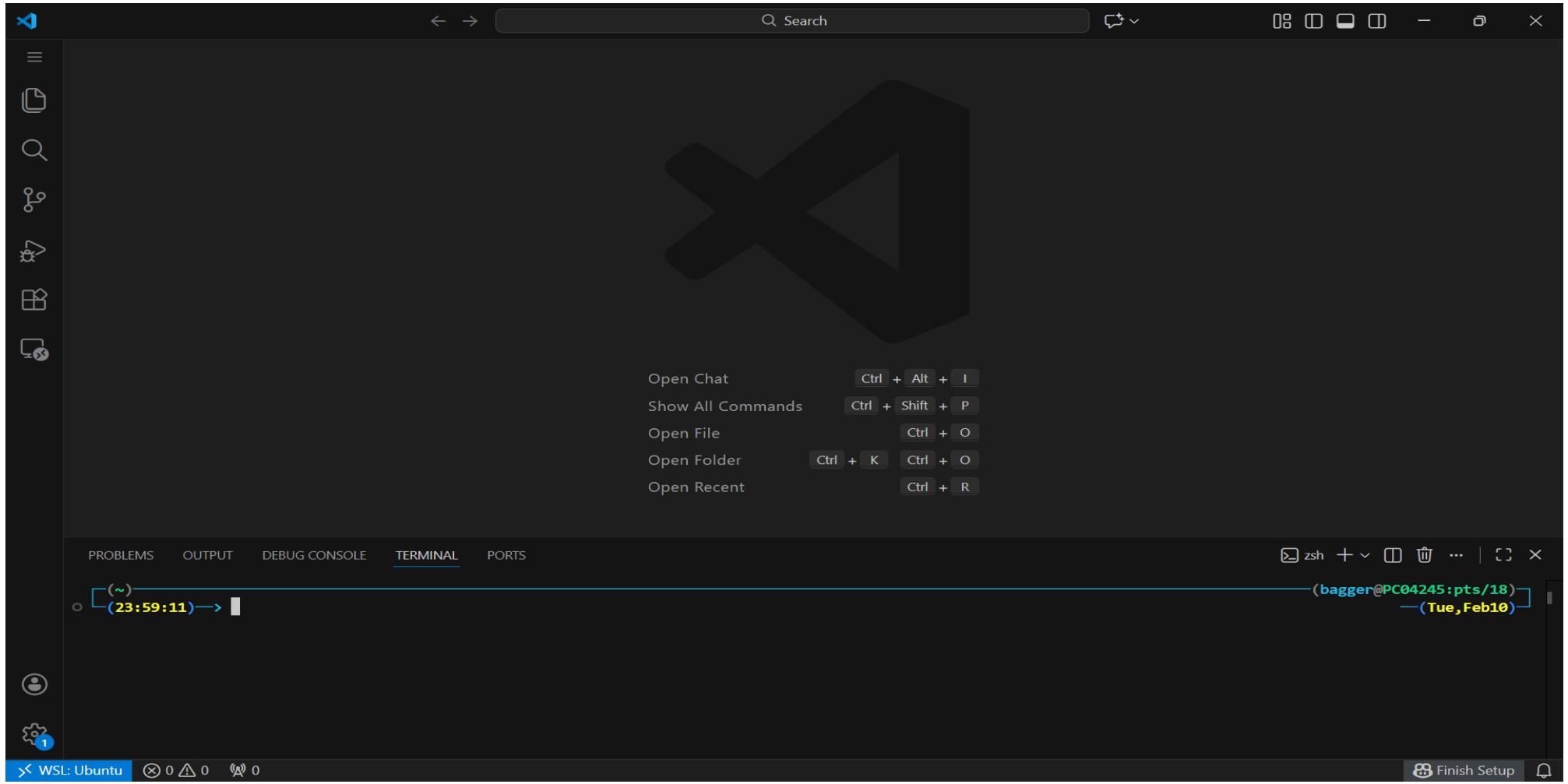
Open Integrated Terminal

Three ways:

1. **Menu:** Terminal → New Terminal
2. **Keyboard:** `Ctrl+` ` (backtick)
3. **Command Palette:** `Ctrl+Shift+P` → "Terminal: Create New Terminal"


Search
File
Edit
Selection
View
Go
Run
Terminal
Help
New Terminal Ctrl+Shift+æ
Split Terminal Ctrl+Shift+5
New Terminal Window Ctrl+Shift+Alt+æ
Run Task...
Run Build Task... Ctrl+Shift+B
Run Active File
Run Selected Text
Show Running Tasks...
Restart Running Task...
Terminate Task...
Configure Tasks...
Configure Default Build Task...
Open Folder Ctrl + K Ctrl + O
Open Recent Ctrl + R
WSL: Ubuntu
Finish Setup

Terminal Appears at Bottom



You'll see:

- Your current directory
- Command prompt
- Shell type (PowerShell/bash/zsh)

This is your connection to the system!

Understanding: Where Am I?

Terminal shows current directory:

```
# Windows
PS C:\Users\YourName>

# Mac/Linux
~ $
```

Check location: `pwd` (print working directory)

Part 4: Installing UV

Install UV from Terminal

Windows (PowerShell):

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Mac/Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Press Enter and wait...

Verify Installation

Close and reopen terminal, then:

```
uv --version
```

Expected: `uv 0.x.x`

✅

If you see version number, UV is installed!

Part 5: Creating Your First Project

Choose Project Location

Navigate to where you want your projects:

This could be e.g. `Documents/`:

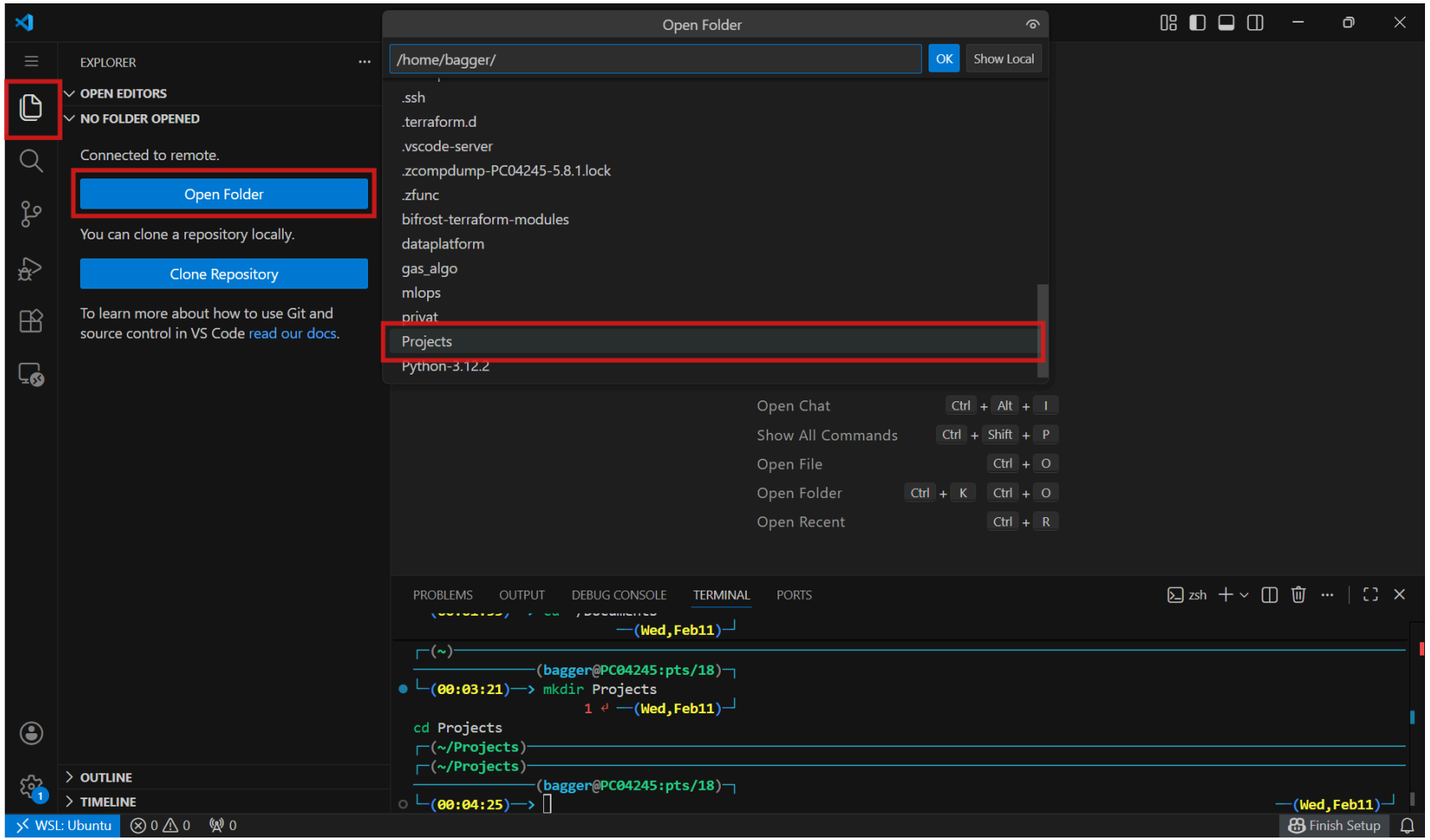
```
# Windows
cd C:\Users\YourName\Documents

# Mac/Linux
cd ~/Documents
```

Create Projects Folder (Optional)

```
mkdir Projects
cd Projects
```

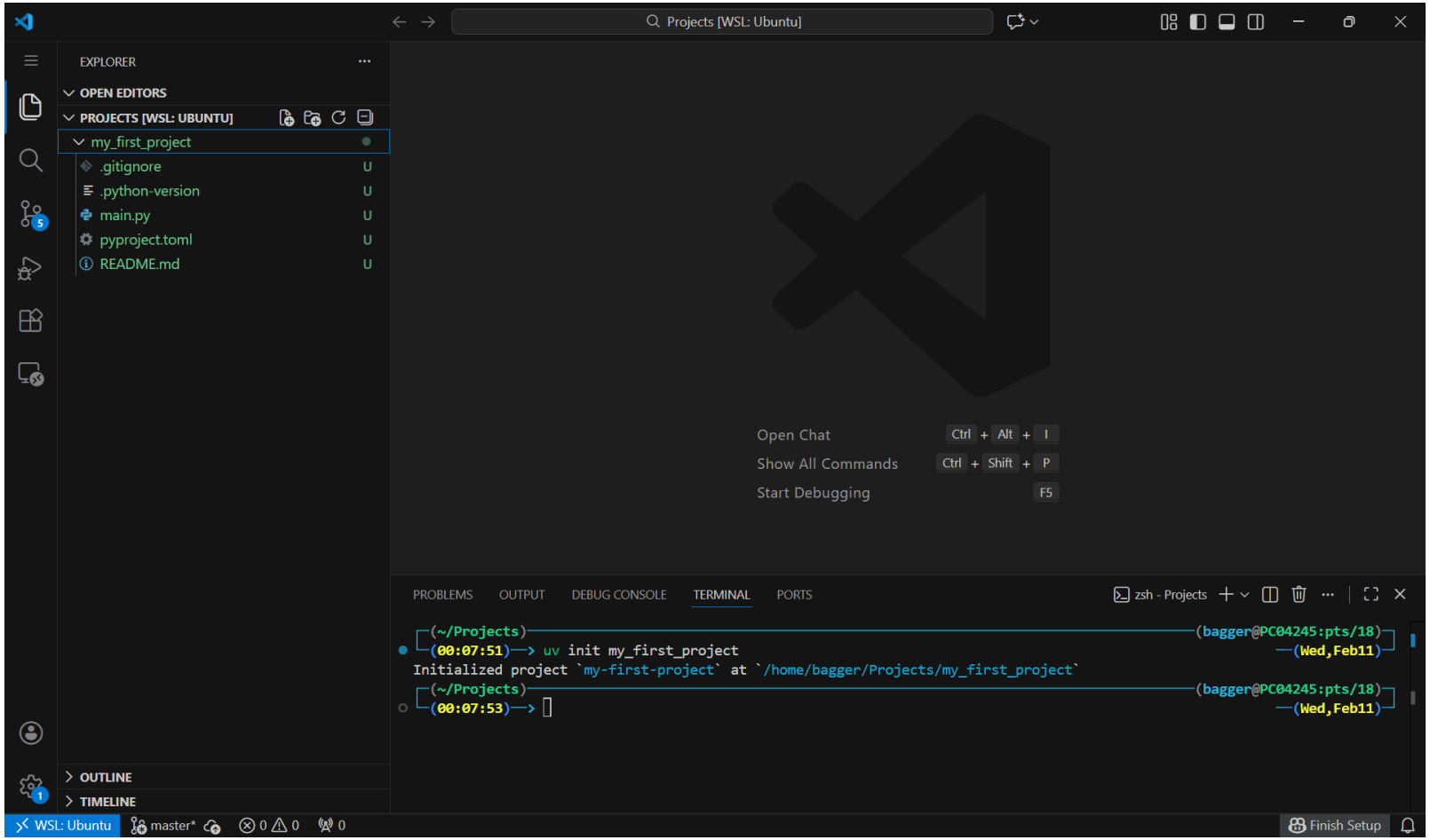
Now you have a clean place for all your projects!

Open folder in VS Code File Explorer



Initialize New Project with UV

```
uv init my_first_project
```



What Just Happened?

UV creates a new folder `my_first_project/` with:

```
my_first_project/
├── pyproject.toml       ← Project settings & dependencies
├── .python-version      ← Python version specification
├── main.py              ← Sample Python file
├── README.md            ← Documentation
└── .gitignore           ← Git ignore rules
```

VS Code File Explorer

Left sidebar shows project files:

You can now see:

- `pyproject.toml`
- `.python-version`
- `main.py`
- `README.md`
- `.gitignore`

Click any file to open it!

Connection: Explorer <-> Terminal

Explorer shows folders <-> Terminal navigates folders

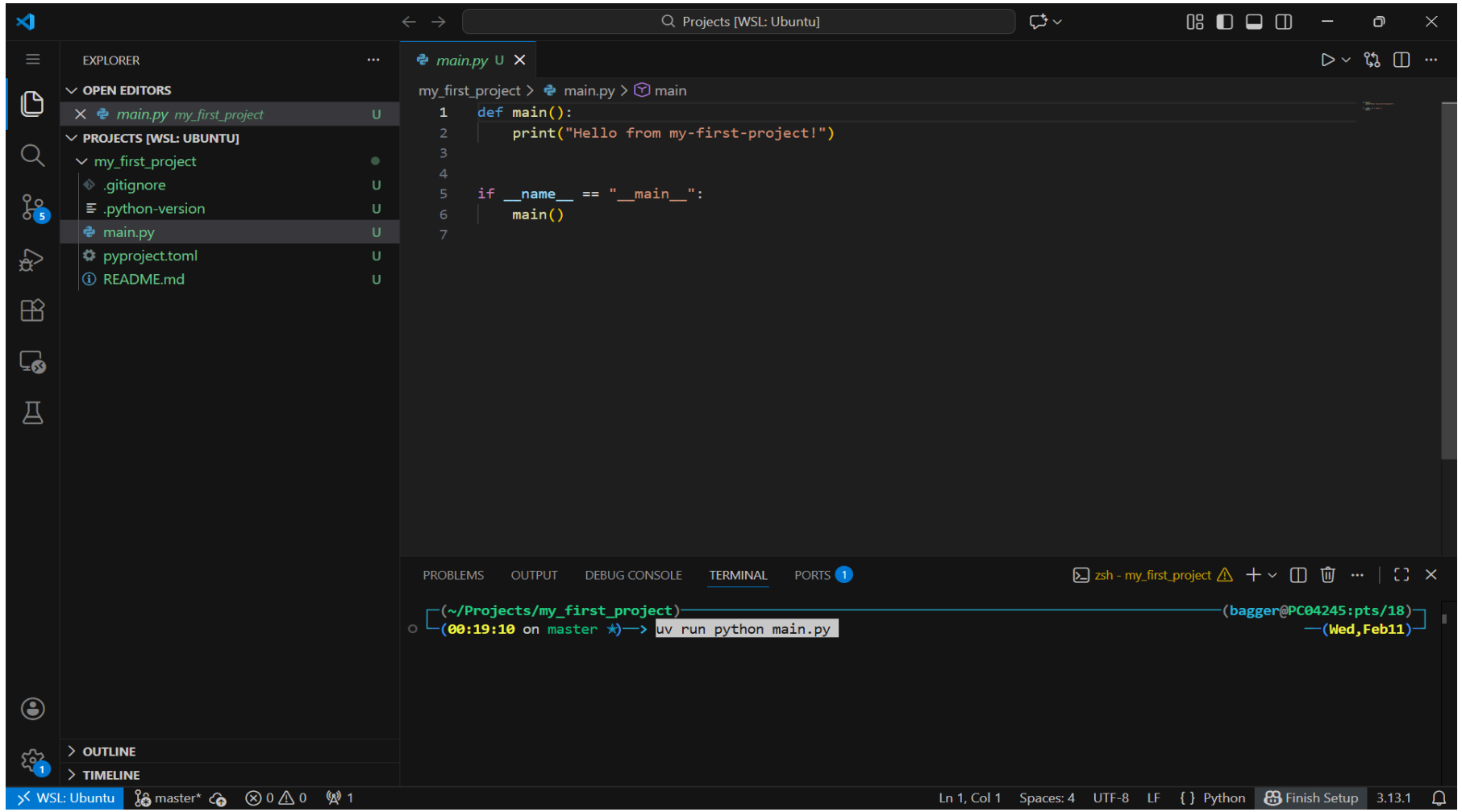
When you:

- **Click folder** in Explorer → See its contents
- `cd folder` in terminal → Navigate to that folder

They're connected to the same location!

Open main.py

Click `main.py` in Explorer:

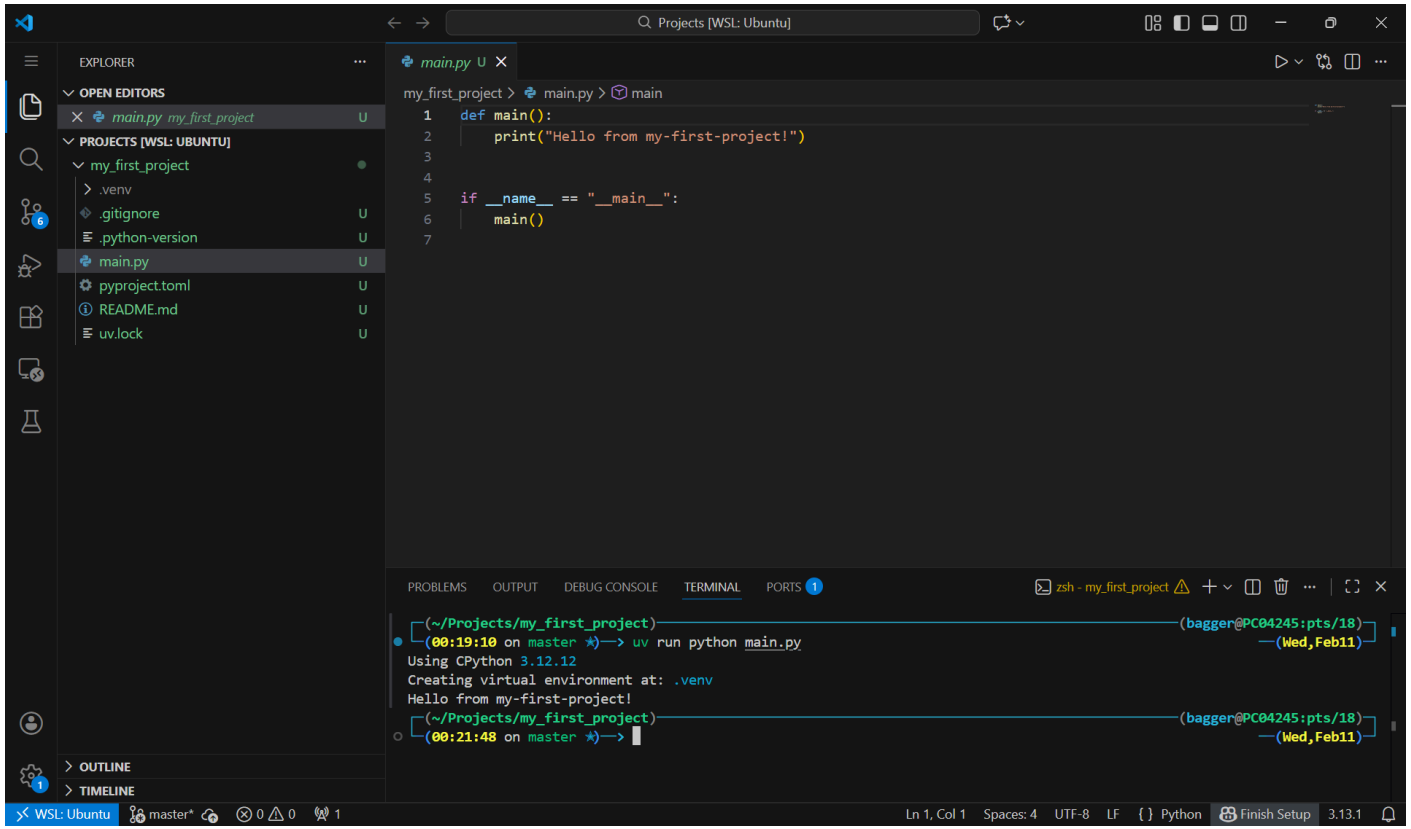


Editor shows the code:

```
def main():
    print("Hello from my_first_project!")

if __name__ == "__main__":
    main()
```

Run Your First Python File (terminal)

```
cd my_first_project # Make sure you're in the project folder
uv run python main.py
```



Output:

```
Hello from my_first_project!
```

🎉

Your first Python program is running!

Understanding: uv run

Why `uv run python main.py`?

```
uv run python main.py
  │      │       │
  │      │       └─ File to run
  │      └─ Python interpreter
  └─ Use project's virtual environment
```

Benefits:

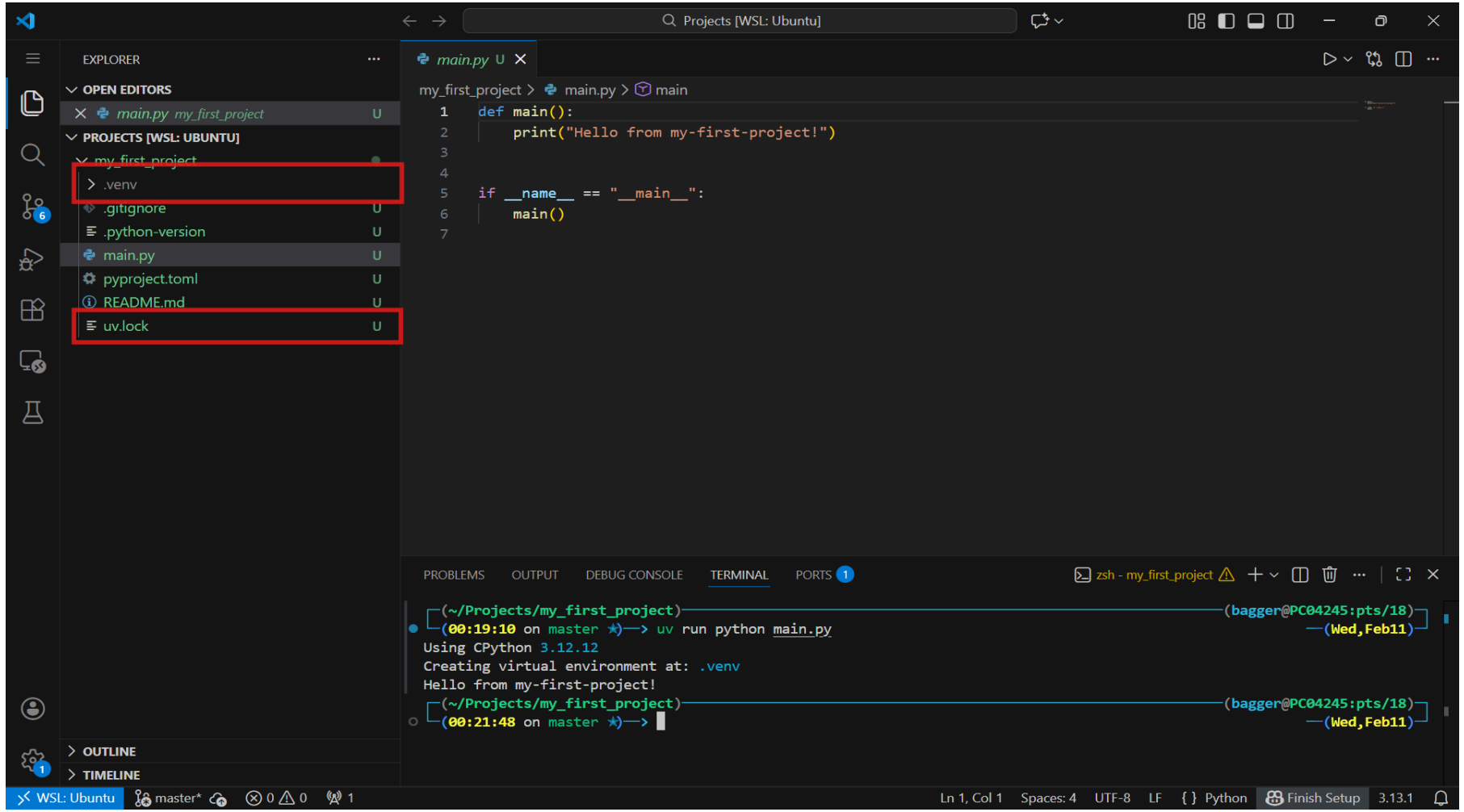
- No manual environment activation needed
- Always uses correct project environment
- Ensures consistency

First `uv run` on a Project

First time you run `uv run` in a project folder, UV will:

1. Create a virtual environment in `.venv/`
2. Install Python (if not already)
3. Create a `uv.lock` file to track dependencies

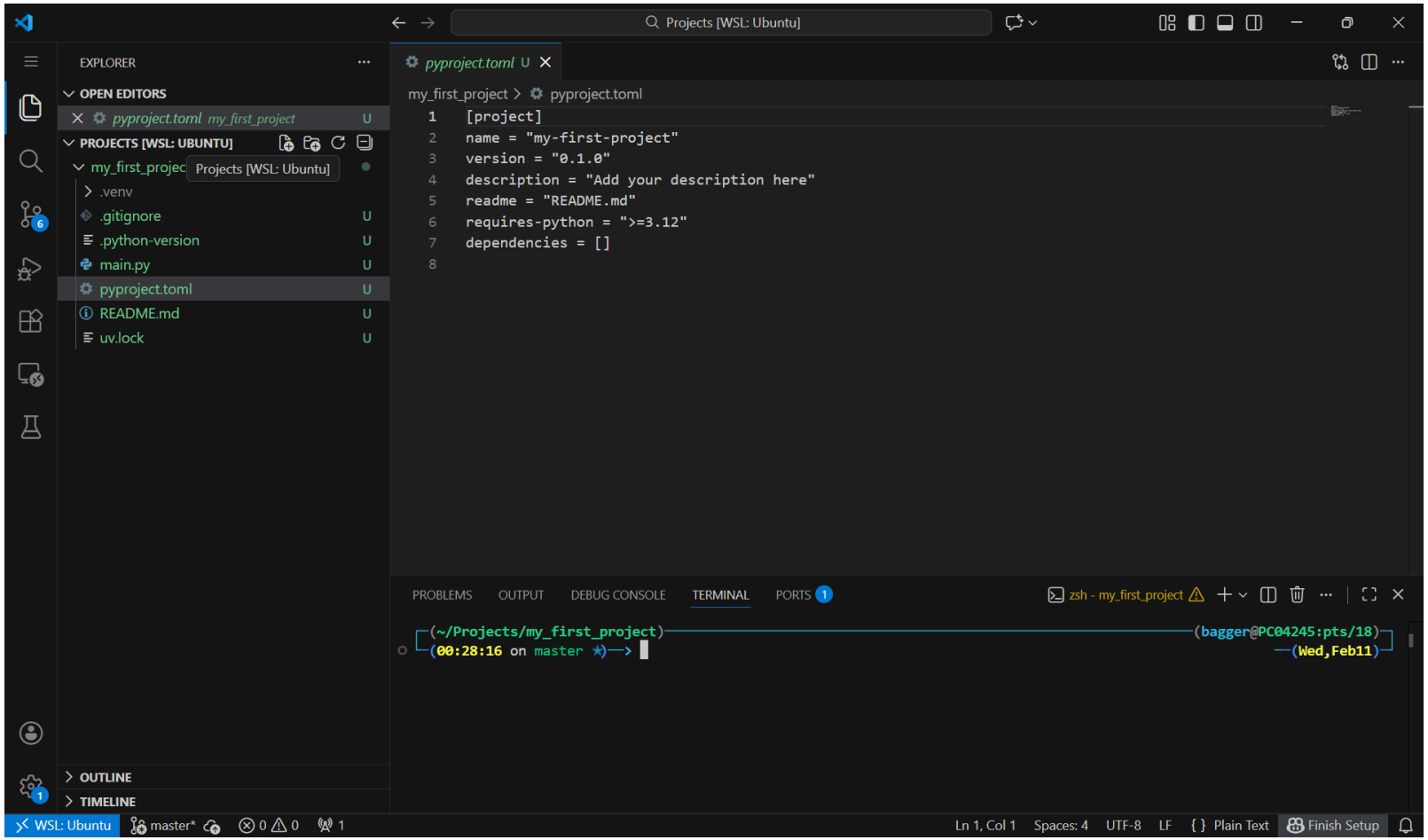
After this, every `uv run` will use that environment!


Projects [WSL: Ubuntu]
EXPLORER
OPEN EDITORS
main.py my_first_project
PROJECTS [WSL: UBUNTU]
my_first_project
.venv
.gitignore
.python-version
main.py
pyproject.toml
README.md
uv.lock
OUTLINE
TIMELINE
main.py
my_first_project > main.py > main
def main():
print("Hello from my-first-project!")
if __name__ == "__main__":
main()
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh - my_first_project
(~/Projects/my_first_project)
(bagger@PC04245:pts/18)
(00:19:10 on master ★)—> uv run python main.py
(Wed,Feb11)
Using CPython 3.12.12
Creating virtual environment at: .venv
Hello from my-first-project!
(~/Projects/my_first_project)
(bagger@PC04245:pts/18)
(00:21:48 on master ★)—>
(Wed,Feb11)
WSL: Ubuntu
master*
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Finish Setup
3.13.1

Part 7: Adding Packages

the pyproject.toml File

Open `pyproject.toml` in Explorer:



This file contains:

The project's configuration:

- Project metadata (name, version)
- Python version requirement
- Dependencies (packages your project needs)
- Development dependencies (testing, formatting tools)
- Build system requirements
- Optional settings for tools

Add Your First Package

Let's add pandas (data analysis library):

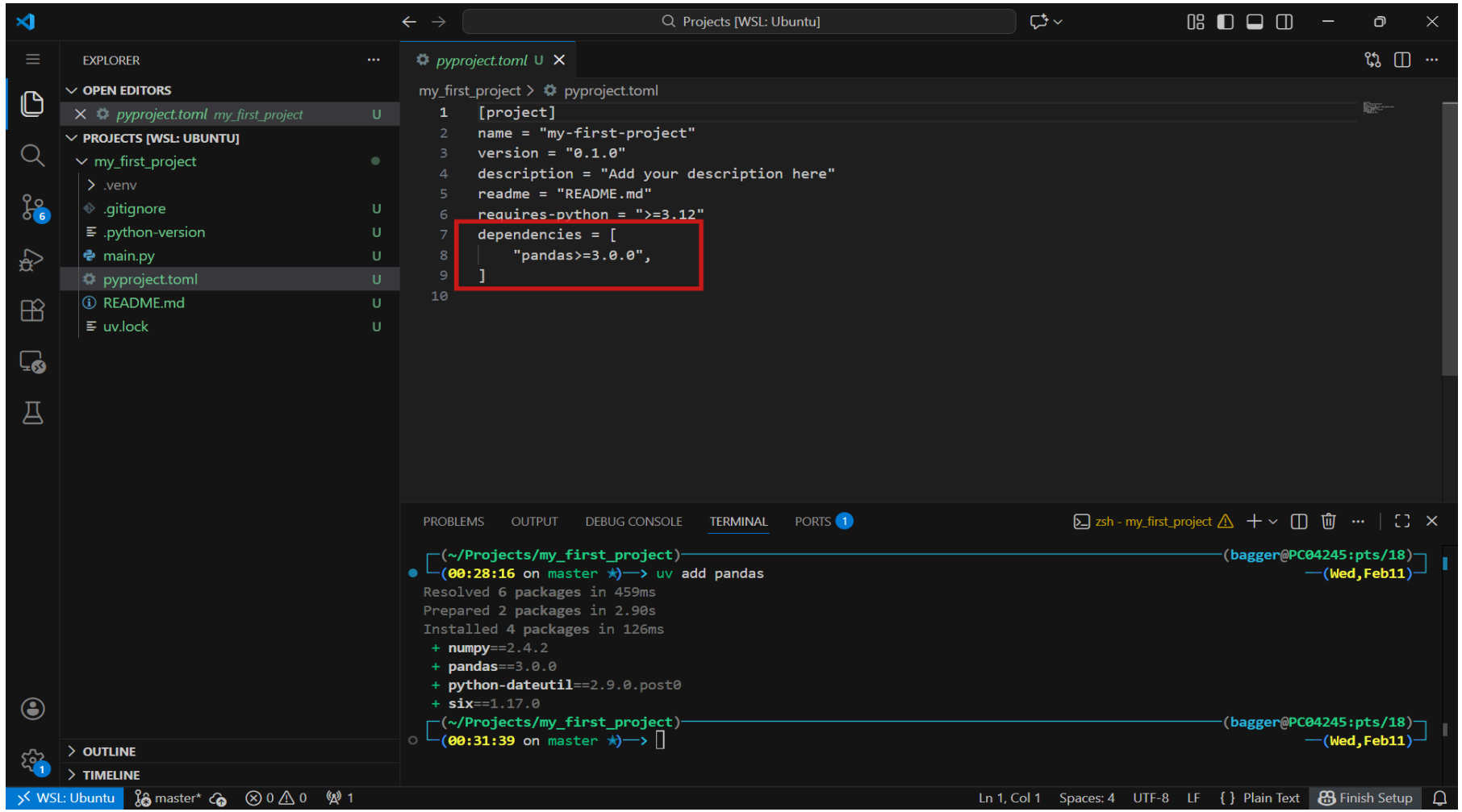
```
uv add pandas
```

UV will:

1. Download pandas
2. Install to `.venv/`
3. Update `pyproject.toml`
4. Create or update `uv.lock` (lock of exact versions for reproducibility!)

See What Changed

Open `pyproject.toml` in Explorer:



Extra

Create a Data Analysis Script

Click "New File" button in Explorer:

Name it: `analyze.py`

Write Code in Editor

Type in `analyze.py`:

```
import pandas as pd

# Create simple dataset
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['Aarhus', 'Copenhagen', 'Odense']
}

df = pd.DataFrame(data)
print(df)
```

Run Your Analysis

In terminal:

```
uv run python analyze.py
```

Output:

```
      Name  Age        City
0    Alice   25      Aarhus
1      Bob   30  Copenhagen
2  Charlie   35      Odense
```

Understanding the Workflow

The Complete Flow

1. **File Explorer** (Left)
   - See all project files
   - Click to open/edit
2. **Editor** (Center)
   - Write and edit code
   - Multiple files in tabs
3. **Terminal** (Bottom)
   - Install packages (`uv add`)
   - Run code (`uv run python script.py`)
   - Navigate folders (`cd`)

Workflow Example: Adding a Feature

1. **Install package** (Terminal):

```
uv add matplotlib
```

2. **Create file** (Explorer):

- Click "New File" → `plot.py`

3. **Write code** (Editor):

```
import matplotlib.pyplot as plt
# ... your plotting code
```

4. **Run** (Terminal):

```
uv run python plot.py
```

File Explorer
↔️
Terminal Navigation

They show the same location!

Explorer folder tree = Terminal `cd` path

When you:

- Click folder in Explorer → Terminal still at same place
- `cd folder` in terminal → Explorer shows you're there
- `cd ..` (go up) → Explorer shows parent folder

Common Tasks

Task 1: Create New Python File

Explorer:

1. Click "New File" button
2. Name: `my_script.py`

Terminal:

```
# File now exists!
uv run python my_script.py
```

Task 2: Create New Folder

Explorer:

1. Right-click in file area
2. "New Folder"
3. Name: `data/`

Terminal:

```
# Or create from terminal
mkdir data
cd data
```

Task 3: Install Multiple Packages

Terminal:

```
uv add numpy pandas matplotlib
```

See changes in:

- `pyproject.toml` (dependencies list)
- `uv.lock` (exact versions)

Task 4: Open Another Project

File → Open Folder...

Or from terminal:

```
cd ../another_project
code .
```

Tips & Tricks

Tip 1: Integrated Terminal Follows Explorer

When you open a new folder:

- Terminal **automatically** changes to that folder
- Always in sync with current project!

Tip 2: Multiple Terminal Windows

Click `+` in terminal panel:

Use for:

- One for running code
- One for installing packages
- One for Git commands

Tip 3: Command Palette

Press: `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (Mac)

Search for any command:

- "Python: Select Interpreter"
- "Terminal: Create New Terminal"
- Anything!

Tip 4: Quick File Navigation

Press: `Ctrl+P` / `Cmd+P`

Type filename → Opens instantly!

No clicking through folders!

Tip 5: Terminal Shortcuts

```
# Navigation
pwd          # Print current directory
ls           # List files (Mac/Linux)
dir          # List files (Windows)
cd ..        # Go up one level
cd ~         # Go to home directory

# Clear screen
clear        # Mac/Linux
cls          # Windows
```